Love of Variety Model and Simulation

Tate Mason

March 28, 2026

1 Model

1.1 Consumer's Problem

The consumer's problem is to maximize utility subject to their choice set. The utility function is as follows:

$$U_{ijt} = \beta_i X_{jt} - (\gamma_i \Sigma_j)^2 + \zeta_{jt} a + \xi_j + \epsilon_{ijt} \tag{1}$$

Where U_{ijt} is the utility of consumer i from product j at time t. X_{jt} is the attribute of product j at time t. γ_i denotes a cost of sameness. Σ_j is the sum of product attributes chosen until time t. Note, γ_i has a standard normal distribution. ζ_{jt} captures some utility from advertising on product j at time t. ξ_j captures utility from unobserved quality. Finally, ϵ_{ijt} is a random error term distributed T1EV.

$$P_{ijt} = \frac{e^{U_{ijt}}}{1 + \sum_k e^{U_{ikt}}} \tag{2}$$

Where the denominator sums over all products k in the choice set at time t, with the outside option.

1.2 Firm's Problem

The firm will maximize profits by choosing how to advertise to the consumer according to their perceptions surrounding the distribution of γ.

$$\pi = (1 + a)p - c \tag{3}$$

Where a is the advertising level and p is the price of the product, and c is some baseline cost for the firm. If the firm chooses not to advertise, they get a profit of $p - c$. If they choose to advertise, they get a profit of $(1 + a)p - c$.

The firm can choose to advertise at different levels, such that $a \in [0, 1]$. This can be conceived of as a loss of revenue in hopes of maintaining a customer. The firm chooses a to maximize profits based on the following function:

$$V(a|\lambda, \Sigma_j, a_{t-1}) = \max_{\pi} \{\pi + \int_{\gamma} V'(a'|\lambda, \Sigma_j, a) d\lambda(\gamma)\} \tag{4}$$

Here, $\lambda(\gamma)$ is the firm's belief about the distribution of γ in the consumer population. Σ_j is the sum of product attributes chosen until time t. a_{t-1} is the advertising level at time $t - 1$. The firm chooses a to maximize profits based on its current beliefs and the expected future value of advertising. Note, the distribution of $\lambda(\gamma) \sim N(\frac{1}{\Sigma_j}, \sigma_\lambda)$. This states they use past purchases to update their mean belief about the distribution of γ, but they have some variance in that belief.

2 Firm Optimization

The firm will choose a to maximize profits π based on the perceived distribution of γ in the consumer population. The firm will also keep track of the sum of product attributes chosen by the consumer, Σ_j, to update their beliefs about γ. The firm will solve the following optimization problem:

$$V(a, \Sigma_j|\lambda(\gamma)) = \max_a\{\pi(a) + \int_\gamma V(a', \Sigma_j'|\lambda(\gamma))d\lambda(\gamma)\} \tag{5}$$

Where $\pi(a) = (1+a)p - c$ is the profit function, $\lambda(\gamma) \sim N(\frac{1}{\Sigma}, \sigma_\lambda)$, and $\Sigma_j' = \Sigma_j + X_{jt} \cdot P_{ijt}$ is the updated sum of product attributes chosen by the consumer conditional on choosing product j at time t. The firm's FOC's are as follows:

$$\begin{aligned}
\frac{\partial V}{\partial a} &= p + P_{ijt}(1 - P_{ijt}) \int_\gamma [V(\Sigma_j + X_{jt}|\lambda'(\gamma)) - V(\Sigma_j|\lambda'(\gamma))]d\lambda(\gamma) = 0 \\
&= p + P_{ijt}(1 - P_{ijt})[V(\Sigma_j + X_{jt}|\lambda(\Sigma_j + X_{jt})) - V(\Sigma_j|\lambda(\Sigma_j))] = 0 \\
&\Rightarrow p^* = (P_{ijt}(1 - P_{ijt})) \cdot [V(\Sigma_j + X_{jt}|\lambda(\Sigma_j + X_{jt})) - V(\Sigma_j|\lambda(\Sigma_j))]
\end{aligned}$$

The firm will choose a such that the marginal profit from advertising equals the expected marginal value of advertising in terms of future profits. The firm will update their beliefs about γ based on the consumer's choices and the sum of product attributes chosen, Σ_j.

3 Results

The simulation will involve setting β and γ at differing levels to observe how consumer choices change.

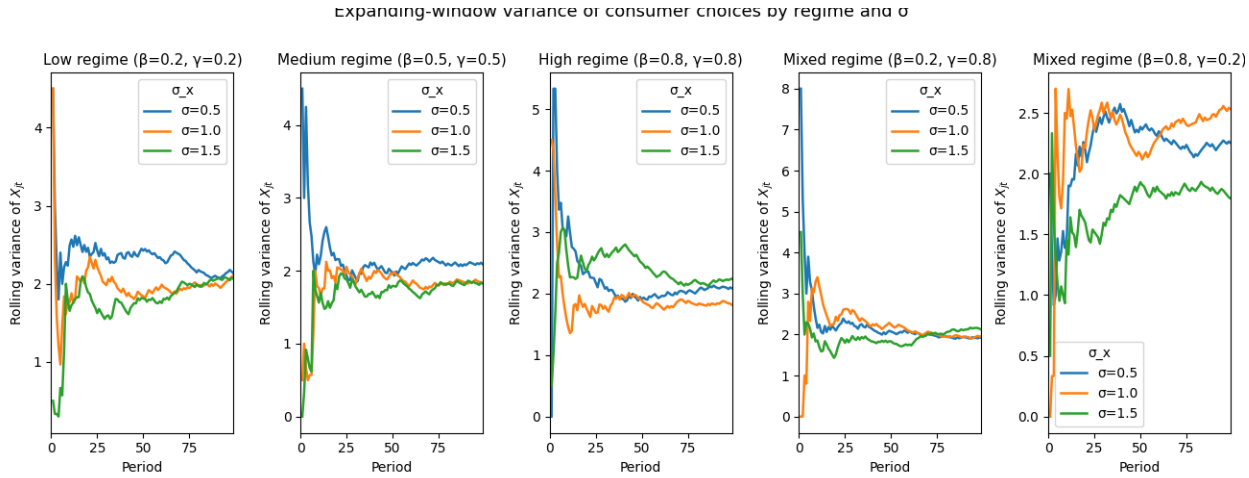
Further, we will look at the basics of the firm's decision of what to present to the consumer. The firm will "send" three products each period. Define $C_t = \{c_1, c_2, c_3\}$ as the set of products the firm sends at time t. $C_t \sim N(\bar{X_{j,t,x}})$ where $\bar{X_{j,t}}$ is the average attribute of product j at time t and σ_x is the standard deviation of the attributes of the products sent by the firm. We want to see how the firm will react to various regimes in terms of σ_x and how that will affect consumer choices.

3.1 Consumer Choices and Utility

The blue line represents the chosen product attribute at time t. The orange line represents the utility from that period. The black dashed line is the evolution of the mean attribute. From the figures, we can see that in the low regime, it seems that after about 50 periods, we converge to a "steady state" where utiility is in line with the chosen attribute, implying in this case they have been able to find the "sweet spot". In the medium regime, It takes closer to 75 periods, with a lot of volatility even until the 70th or so period. In this period the "sweet spot" is less clear, as there is still lower utility than attribute value. In the high regime, we actually see a decent fit, but with spikes intermittently. This is likely because when the consumer is more sensitive to variety, the deviations will be more pronounced. They also like what they like more, so this creates an interesting dynamic where they want newness (which they may like), but if it is too new they'll be punished. In the regime with low utility from characteristic and a higher variety parameter, we cannot get a good fit, with utility going all over the place, especially in a negative direction. This is likely because the consumer is very sensitive to newness, and any new draw of σ_x that is too high will cause a sharp drop in utility. In the inverse regime, we see almost the exact opposite, though with an admittedly better fit to the X_{jt} chosen. Here, this is likely because the consumer is just happy to have new product characteristics, and is relatively insensitive to redundancy in product characteristics. This is why we see positive spikes in utility.

All in all, we see that utility is more volatile as X_{jt} deviates from $\bar{X}_{jt}$, a data fact we should expect. We also see that the initial parameters of the consumer have a large effect of the model.

3.2 Rolling Variance of Consumer Choices and σ_x



Now, we can discuss the rolling variance of σ_x. I set it to 3 starting values $\sigma_x = [0.5, 1.0, 1.5]$. Starting in the low regime, the high value of σ_x grows in variance over the entire time horizon while the middle and low value reach steady state after about 40-50 periods. In the medium regime, they converge similarly, with $\sigma_x = 1.5$ having decreasing variance for the first 10 or so periods. The middle and low values do not really evolve, staying fairly constant over the time horizon. In the high regime, All three values grow in variance, but the high value grows the entire time while the other two converge after about 25-35 periods. In the regime with $\beta = 0.2, \gamma = 0.8$, the high value of the high value grows the entire time, while the other two almost immediately converge to a steady state. In the inverse regime, the high variance falls immediately, before climbing back for about 10 periods, then falling for 20 periods and then converging. The other two converge after about 10 periods.

Here, it seems that without high variance, sigma is not super volatile regardless of regime. When variance is high, however, it is very volatile with respect to γ especially. This is likely because when the consumer is more sensitive to variety, deviations will be more pronounced and so the variance of σ_x will be more volatile.

4 Code

```
import pandas as pd
import numpy as np
import scipy as sp
import matplotlib.pyplot as plt
import seaborn as sns

"""
This␣file␣will␣be␣used␣to␣demonstrate␣the␣initial␣simulation␣of␣the␣consumer's␣problem
    ␣over␣100␣periods.␣We␣will␣use␣one␣consumer␣to␣start,␣with␣a␣pre-determined
set␣of␣parameters:
␣␣␣␣-␣beta_low␣=␣0.2
```

```
␣␣␣␣-␣beta_high␣=␣0.8
␣␣␣␣-␣beta␣=␣0.5
␣␣␣␣-␣gamma_low␣=␣0.2
␣␣␣␣-␣gamma_high␣=␣0.8
␣␣␣␣-␣gamma␣=␣0.5
␣␣␣␣-␣X_jt␣=␣5
␣␣␣␣-␣a_ijt␣~␣N(alpha*(1/X_jt-X_bar),␣sigma_a)␣where␣alpha␣is␣set␣in␣different␣regimes
We␣will␣simulate␣the␣consumer's␣choices␣over␣100␣periods,␣and␣then␣analyze␣how␣the␣
    objective␣changes␣over␣time,␣how␣the␣variance␣in␣choices␣change,␣and␣how␣the
variance␣changes␣with␣regimes.␣We␣also␣want␣to␣see␣how␣X␣and␣sigma␣comingle.

␣␣␣␣Our␣objective␣function␣is:
␣␣␣␣U_jt␣=␣beta_i*X_jt␣-␣gamma*(X_jt␣-␣x_bar)^2␣+␣pi*a_ijt␣+␣epsilon_ijt
␣␣␣␣where␣epsilon_ijt␣~␣T1EV

␣␣␣␣P_ijt␣=␣int␣exp(U_ijt)␣/␣1␣+␣sum_k␣exp(U_ikt)␣dF_j
␣␣␣␣s_jt␣=␣int␣P_ijt␣dF_j
␣␣␣␣HHI_t␣=␣sum_j␣s_jt^2
␣␣␣␣-␣note:␣individual␣HHI,␣such␣that␣HHI␣is␣the␣share␣of␣chosen␣product␣squared,␣not␣
    market␣share␣squared.
"""

rng = np.random.default_rng(seed=219) # set seed for reproducibility

# Parameters
T = 100
sigma_values = [0.5, 1.0, 1.5] # different variance levels

# Regimes
regimes = [
    (0.2, 0.2, 'Low␣regime␣(  =0.2,␣  =0.2)'),
    (0.5, 0.5, 'Medium␣regime␣(  =0.5,␣  =0.5)'),
    (0.8, 0.8, 'High␣regime␣(  =0.8,␣  =0.8)'),
    (0.2, 0.8, 'Mixed␣regime␣(  =0.2,␣  =0.8)'),
    (0.8, 0.2, 'Mixed␣regime␣(  =0.8,␣  =0.2)')
]

def simulate(beta, gamma, sigma, T, rng):
    options = np.array([1.0, 2.0, 3.0, 4.0, 5.0])

    x_choices = np.zeros(T)
    x_bar = np.zeros(T)

    x_choices[0] = rng.choice(options)
    x_bar[0] = x_choices[0] # initial mean is just the first choice
    for t in range(1, T):
        x_bar[t] = np.mean(x_choices[:t]) # mean of all prior choices
        x_choices[t] = rng.choice(options)
```

```
    return x_choices, x_bar


def rolling_var(x, min_periods=2):
    var = np.full(len(x), np.nan)
    for t in range(min_periods, len(x) + 1):
        var[t-1] = np.var(x[:t], ddof=1)
    return var

fig, axes = plt.subplots(1, 5, figsize=(13, 5), sharey=False)
idx = np.arange(T)
colors = ["#1f77b4", "#ff7f0e", "#2ca02c"]  # one per sigma value

for ax, (beta, gamma, label) in zip(axes, regimes):
    for sigma, color in zip(sigma_values, colors):
        x_choices, _ = simulate(beta, gamma, sigma, T, rng)
        roll_var = rolling_var(x_choices)
        ax.plot(idx, roll_var, label=f"  ={sigma}", color=color, linewidth=1.8)

    ax.set_title(label, fontsize=11)
    ax.set_xlabel("Period")
    ax.set_ylabel("Rolling␣variance␣of␣$X_{jt}$")
    ax.legend(title=" _x ")
    ax.set_xlim(0, T - 1)

fig.suptitle("Expanding-window␣variance␣of␣consumer␣choices␣by␣regime␣and␣  ",
    fontsize=13, y=1.01)
plt.tight_layout()
plt.savefig("../Output/rolling_variance_of_consumer_choices_by_regime_and_sigma.png")
plt.show()

# Consumer Choices

C_x = "#4C72B0"
C_Xb = "black"
C_U = "#C44E52"

fig2, axes2 = plt.subplots(1, 5, figsize=(16, 5), sharey=False)

for ax, (beta, gamma, label) in zip(axes2, regimes):
    x_choices, x_bar = simulate(beta, gamma, sigma_values[1], T, rng)
    epsilon = rng.gumbel(size=T)
    U = beta * x_choices - gamma * (x_choices - x_bar) ** 2 + epsilon
    x_smooth = np.convolve(x_choices, np.ones(5)/5, mode='same')
    ax.plot(idx, x_smooth, label="$X_{jt}$", color=C_x, linewidth=1.8, alpha=0.7)
    ax.plot(idx, x_bar, label="$\\bar{X}_{jt}$", color=C_Xb, linewidth=1.2, alpha=0.7,
         linestyle='--')
```

```
    U_smooth = np.convolve(U, np.ones(5)/5, mode='same')
    ax.plot(idx, U_smooth, label="$U_{jt}$", color=C_U, linewidth=1.2, alpha=0.6)
    ax.set_title(label, fontsize=10)
    ax.set_xlabel("Period")
    ax.set_ylabel("Value")
    ax.legend(fontsize=9)
    ax.set_xlim(-1, T)
    ax.set_ylim(-3, 5)
fig2.suptitle("Consumer choices and utility by regime (  =1.0)", fontsize=13, y=1.01)
plt.tight_layout()
plt.savefig("../Output/consumer_choices_and_utility_by_regime.png")
plt.show()

# Calculating Choice Probabilities and HHI

options = np.array([1.0, 2.0, 3.0, 4.0, 5.0]) # possible choices

beta, gamma = 0.5, 0.5
x_choices, x_bar = simulate(beta, gamma, sigma_values[1], T, rng)  # using medium
    sigma for choice simulation

j_obs = np.searchsorted(options, x_choices) # observed choice indices
eps = rng.gumbel(size=(5, T)) # random shocks for each option and period

def calculate_prob_and_hhi(options, x_bar, beta, gamma, eps, j_obs):
    T = len(x_bar)
    ll = 0.0
    s_all = np.zeros((T, len(options)))
    for t in range(T):
        V = beta * options - gamma * (options - x_bar[t]) ** 2 + eps[:, t]
        log_P = V - sp.special.logsumexp(V)
        ll += log_P[j_obs[t]]
        s_all[t] = np.exp(log_P)
    HHI = np.mean(np.sum(s_all ** 2, axis=1))
    return s_all, HHI, ll
```